LAB MANUAL

PART A

(PART A: TO BE REFERRED BY STUDENTS)

Experiment No: -8

Aim:- Implementation and analysis of the Longest Common Subsequence (LCS) problem using the Dynamic Programming approach.

Resource Required:- Any C compiler, VSCode or online compiler (e.g., OnlineGDB, Code::Blocks)

Theory:-

Dynamic Programming Approach:

1. Dynamic Programming solves a complex problem by dividing it into smaller overlapping subproblems, solving each subproblem once, and storing its result for reuse.
1. It is suitable for problems having optimal substructure and overlapping subproblems. Longest Common Subsequence is a classic Dynamic Programming problem.

Longest Common Subsequence (LCS):

1. Given two sequences, the Longest Common Subsequence is the longest sequence that appears in both sequences in the same relative order, but the characters need not be contiguous.
1. For example, for X = "AGGTAB" and Y = "GXTXAYB", one Longest Common Subsequence is "GTAB", whose length is 4.
1. Let L[i][j] represent the length of the LCS of the first i characters of X and the first j characters of Y.
1. If the current characters are equal, they contribute one character to the LCS. Otherwise, the maximum LCS obtained by excluding one character from either sequence is selected.

Recurrence Relation:

L[i][j] = 0 if i = 0 or j = 0

L[i][j] = 1 + L[i-1][j-1] if X[i-1] = Y[j-1]

L[i][j] = max(L[i-1][j], L[i][j-1]) if X[i-1] != Y[j-1]

Algorithm:-

```
LCS(X, Y, m, n)
   create table L[0...m][0...n]
   for i = 0 to m
      for j = 0 to n
         if i == 0 or j == 0
            L[i][j] = 0
         else if X[i-1] == Y[j-1]
            L[i][j] = L[i-1][j-1] + 1
         else
            L[i][j] = max(L[i-1][j], L[i][j-1])
   return L[m][n]
```

C Implementation:-

```
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
```

```
    return (a > b) ? a : b;
}

int main() {
    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";

    int m = strlen(X);
    int n = strlen(Y);
    int L[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;
            else
                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
        }
    }

    int index = L[m][n];
    char lcs[index + 1];
    lcs[index] = '\0';

    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (X[i - 1] == Y[j - 1]) {
            lcs[index - 1] = X[i - 1];
            i--;
            j--;
            index--;
        } else if (L[i - 1][j] > L[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }

    printf("First String  : %s\n", X);
    printf("Second String : %s\n", Y);
    printf("Length of LCS : %d\n", L[m][n]);
    printf("LCS           : %s\n", lcs);

    return 0;
}
```

OUTPUT:

First String : AGGTAB

Second String : GXTXAYB

Length of LCS : 4

LCS : GTAB

Dry Run Example — X = "AGGTAB", Y = "GXTXAYB":

	0	**G**	**X**	**T**	**X**	**A**	**Y**	**B**
0	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	4

1. The first row and first column are initialized to 0 because the LCS with an empty sequence has length 0.
1. Whenever X[i-1] and Y[j-1] match, 1 is added to the diagonal value L[i-1][j-1].
1. When the characters do not match, the maximum of the top cell and left cell is stored.
1. The final cell L[6][7] = 4, so the length of the Longest Common Subsequence is 4. Backtracking through the table gives LCS = "GTAB".

Complexity Analysis:

Algorithm	**Best Case**	**Average Case**	**Worst Case**
LCS (Dynamic Programming)	O(mn)	O(mn)	O(mn)

1. The algorithm fills an (m + 1) × (n + 1) Dynamic Programming table. Therefore, the time complexity is O(mn), where m and n are the lengths of the two input sequences.
1. The standard tabulation approach requires O(mn) auxiliary space for the DP table. If only the LCS length is required, space can be optimized to O(min(m,n)); however, reconstructing the actual subsequence generally requires retaining additional information.

Conclusion:

You should have learned how Dynamic Programming is used to solve the Longest Common Subsequence problem, constructed and traced the DP table, obtained the LCS length and subsequence, and analyzed its time and space complexity.

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per the following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in-charge faculty at the end of the practical in case there is no Blackboard access available.)

Roll No.	60	**Name:**	Avdhut Parish Madhavi
Class	C	**Batch:**	C2
Date of Experiment:	30-09-2026	**Date of Submission:**	
Grade:			

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

1. Run the program using X = "ABCDGH" and Y = "AEDFHR". Construct the DP table and determine the LCS and its length.

```
#include <stdio.h>
#include <string.h>
#include <conio.h>
#define Max 100
int max(int a, int b) {
   return (a > b) ? a : b;
}

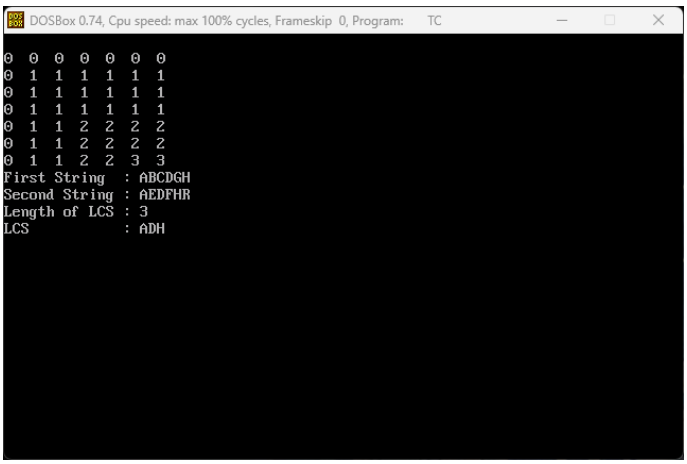
int main() {
   char X[] = "ABCDGH";
   char Y[] = "AEDFHR";
   char lcs[Max];
   int i,j,index=0;
   int m = strlen(X);
   int n = strlen(Y);
   int L[6 + 1][6 + 1];
   clrscr();
   for (i = 0; i <= m; i++) {
         for (j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                  L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                  L[i][j] = L[i - 1][j - 1] + 1;
            else
                  L[i][j] = max(L[i - 1][j], L[i][j - 1]);
         }
   }
   printf("\n");
   for(i = 0; i<=m ;i++)
   {
         for(j=0; j<=n; j++)
         {
                  printf("%d  " ,L[i][j]);
         }
         printf("\n");
   }
   index = L[m][n];
   lcs[index] = '\0';

   i = m, j = n;
   while (i > 0 && j > 0) {
         if (X[i - 1] == Y[j - 1]) {
            lcs[index - 1] = X[i - 1];
            i--;
            j--;
            index--;
         } else if (L[i - 1][j] > L[i][j - 1]) {
            i--;
         } else {
            j--;
         }
   }

   printf("First String  : %s\n", X);
   printf("Second String : %s\n", Y);
   printf("Length of LCS : %d\n", L[m][n]);
   printf("LCS           : %s\n", lcs);

   getch();
   return 0;
}
```

O/P: -



2. Run the program using two different input strings of your choice and verify the result by tracing the Dynamic Programming table.

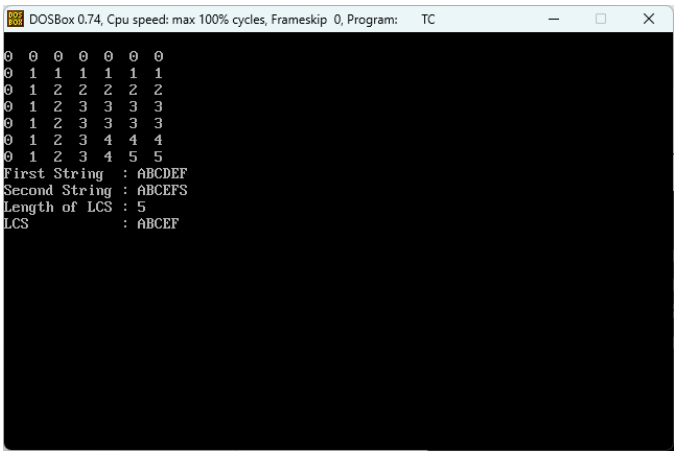
```
#include <stdio.h>
#include <string.h>
#include <conio.h>
#define Max 100
int max(int a, int b) {
   return (a > b) ? a : b;
}

int main() {
   char X[] = "ABCDEF";
   char Y[] = "ABCEFS";
   char lcs[Max];
   int i,j,index=0;
   int m = strlen(X);
   int n = strlen(Y);
   int L[6 + 1][6 + 1];
   clrscr();
   for (i = 0; i <= m; i++) {
    for (j = 0; j <= n; j++) {
       if (i == 0 || j == 0)
         L[i][j] = 0;
       else if (X[i - 1] == Y[j - 1])
         L[i][j] = L[i - 1][j - 1] + 1;
       else
         L[i][j] = max(L[i - 1][j], L[i][j - 1]);
    }
   }
   printf("\n");
   for(i = 0; i<=m ;i++)
   {
    for(j=0; j<=n; j++)
    {
        printf("%d  " ,L[i][j]);
    }
    printf("\n");
   }
   index = L[m][n];
   lcs[index] = '\0';
   i = m, j = n;
   while (i > 0 && j > 0) {
    if (X[i - 1] == Y[j - 1]) {
       lcs[index - 1] = X[i - 1];
       i--;
       j--;
       index--;
    } else if (L[i - 1][j] > L[i][j - 1]) {
       i--;
    } else {
       j--;
    }
   }

   printf("First String  : %s\n", X);
   printf("Second String : %s\n", Y);
   printf("Length of LCS : %d\n", L[m][n]);
   printf("LCS           : %s\n", lcs);

   getch();
   return 0;
 }
```

O/P: -



3. Do the analysis of the Longest Common Subsequence problem using the Dynamic Programming approach.

3. Analysis of LCS:-

Suppose the length of the 2 sequence are m and n

we have a table containing: $(m+1)(n+1)$ cells.

Each cell takes constant time to calculate

Therefore:

$$T(m,n) = O(mn)$$

If both strings are of length n

then, $T(n) = O(n^2)$